How Many Parameters Does it Take to Change a Light Bulb? Evaluating Performance in Self-Play of Conversational Games as a Function of Model Characteristics

Nidhir Bhavsar[1], Jonathan Jordan[1], Sherzod Hakimov[1], David Schlangen[1,2]

[1]Computational Linguistics, Department of Linguistics
University of Potsdam, Germany
[2]German Research Center for Artificial Intelligence (DFKI), Berlin, Germany
firstname.lastname@uni-potsdam.de

arXiv:2406.14051v1 [cs.CL] 20 Jun 2024

Abstract

What makes a good Large Language Model (LLM)? That it performs well on the relevant benchmarks—which hopefully measure, with some validity, the presence of capabilities that are also challenged in real application. But what makes the model perform well? What gives a model its abilities? We take a recently introduced type of benchmark that is meant to challenge capabilities in a goal-directed, agentive context through self-play of conversational games, and analyse how performance develops as a function of model characteristics like number of parameters, or type of training. We find that while there is a clear relationship between number of parameters and performance, there is still a wide spread of performance points within a given size bracket, which is to be accounted for by training parameters such as fine-tuning data quality and method. From a more practical angle, we also find a certain degree of unpredictability about performance across access methods, possible due to unexposed sampling parameters, and a, very welcome, performance stability against at least moderate weight quantisation during inference.

1 Introduction

Previous work has established that LLMs can be made to "self-play" dialogue games, and that their performance in doing so can be used as a differentiator between models (Chalamalasetti et al., 2023; Qiao et al., 2023; Li et al., 2023; Gong et al., 2023; Wu et al., 2023; Zhou et al., 2023; Duan et al., 2024). In particular, Chalamalasetti et al. (2023) have provided construct validity arguments for this approach that connect performance to the underlying constructs *understanding* and *reasoning*. What this previous work has left unexplored is which model properties determine performance. This is what the present paper asks. Our theoretical interest is in the following: What drives performance? What makes a model a good model? (Sections 4, 5). Out of practical interest, and as a guide for researchers interested in tapping the agentive potential of LLMs, we also ask what influences the performance of a given model during inference. (Section 6).

Before we turn to these questions, however, we briefly recap what zero- or few-shot dialogue game play requires from a model.

```
Let us play a game! I will give you a word, and
your task is to describe the concept behind the
word, but without using the word itself and also
without using some other related words that I
give you.
I will then give your description to your
partner. Your partner will guess what the word
was. I will then give you the guess that your
partner made, and if it was wrong, you can give
another clue, again taking care not to use the
forbidden words.
You win if your partner correctly guesses the
word.
Please start your description with DESCRIPTION:,
and produce only one paragraph of text.
The word to describe is {{WORD}}, and the other
forbidden words are {{TABOO}}.
```

Figure 1: Zero-shot prompt template for inducing an agent that plays the 'Taboo' game; modified from Chalamalasetti et al. (2023)

LLMs as Zero-Shot Agents Figure 1 gives an example of how in this approach a game playing agent is induced from an LLM. We can distinguish several distinct elements in this prompt: First, the general goal of the game and the possible moves are explained (first three paragraphs); let us label this as $\mathcal{G}_g$. Second, specific instructions are given on how the response of the model is to be formulated, in order to count as a game move (last paragraph; $\mathcal{G}_f$). We call both together simply $\mathcal{G}$, and its instantiation with a specific goal (here, target and taboo words) $\mathcal{G}_x$.

Let us formalise the underlying decision problem

of which move to take at any point in a Dialogue Game as a Markov Decision Process $(\mathcal{S}, \mathcal{A}, P, R)$, where $\mathcal{S}$ denotes the state of the game, $\mathcal{A}$ the action space, P the transition function $\mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$, and R the reward function $\mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. What we are expecting from the model can then be described as follows: (implicitly) derive P and R from $\mathcal{G}_g$, $\mathcal{A} \in V^{max}$ from $\mathcal{G}_f$ (where V is the model's vocabulary), a policy π $(\mathcal{S} \rightarrow \mathcal{A})$ from $\mathcal{G}_g$ (such that following it optimizes the expected reward), and a *state interpretation function* S that takes the conversation so far $(\mathcal{G}_x, \mathcal{C})$ to a state $s \in \mathcal{S}$. A tall order indeed.

It is in the state interpretation function S and the way it is induced by $\mathcal{G}_g$ (although this is our terminology) that Chalamalasetti et al. (2023) locate the *understanding* abilities of a model (for which they give further analyses into *situation model*, *discourse model*, etc.; see there), and *format instruction following* abilities in the function that applies $\mathcal{G}_f$ to restrict the output space. The performance along these axes is measured by a *quality score* and a *percentage-played score*, respectively, and it is with this fine-grained instrument with which we will relate model characteristics and performance.

2 Related Work

Two lines of work are relevant for our questions. Work on *scaling laws* for neural networks has shown that there is a power law relationship between the size of a neural network model and its performance (Kaplan et al., 2020). While the exact nature of the relationship in this (empirical) "law" is contested (Hoffmann et al., 2022), the general observation has held up well (Gadre et al., 2024). Recent observations point at a larger than previously accounted for the role of the *quality* of the training data (Touvron et al., 2023a,b; Abdin et al., 2024).

With the exception of the most recent work cited above, the work on scaling laws has mostly looked at simple performance measures like the loss on next-token prediction. The second line of work relevant here has, in contrast, been concerned with the dynamics of performance development across training and model parameters. Wei et al. (2022) described patterns that they observed of performance on certain tasks plotted against model size as *emergence* of abilities; an interpretation later supported by Srivastava et al. (2022), and more recently questioned by Lu et al. (2023)

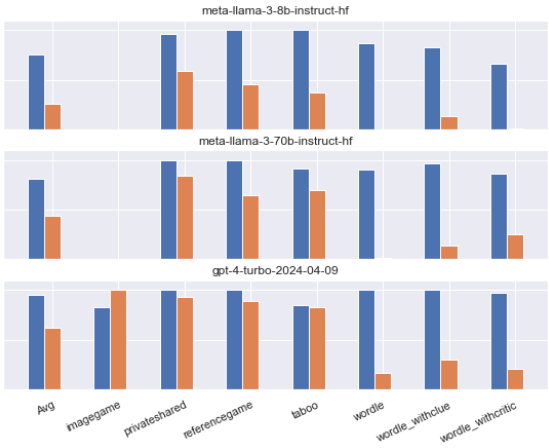

Figure 2: Profiles for three models. For each game (and the average), %-played blue/left, quality brown/right. Top white line is at 100.

Where this work has looked at performance metrics beyond prediction loss or perplexity, it has mostly used classic static NLP tasks. Our interest in the following is to use the recently introduced *game-based evaluation* paradigm, where performance is measured via interactive tasks (in self-play), to investigate empirical relations and development dynamics with respect to parameters such as model size or type of training data and this type of performance.

3 The Starting Point: The Score Matrix

As our measurement instrument, we use the `clembench` of Chalamalasetti et al. (2023), on account of it giving full access to the code, offering access to fine-grained measurements and the distinction between formal instruction following and competence (described above), and it being backed up by validity arguments.[1] An analysis with the same general structure could, of course, be performed using any score-giving benchmark.

We treat `clembench` as a function from model access to a score vector: $\mathcal{C}_{\mathcal{G}_\mathcal{X}} : M \rightarrow \mathbf{s}$ (with each component of $\mathbf{s} \in [0, 1]$), where $\mathcal{C}$ is the `clemgame` framework, $\mathcal{G}_X$ is the set of game definitions and respective game instances, and M is a model interface, in itself consisting of d, the inference/decoding interface, and Θ, the set of weights. For some models, d and Θ cannot vary independently, for access reasons (e.g., there is only one way of addressing GPT-4); for others, they can (e.g., accessing $\Theta_{\text{llama-3-70B-instruct}}$ via an API or a local

[1] Dialogue games contained in `clembench` are `taboo`, `imagegame` (`drawing`), `referencegame`, `wordle`, `wordle with clue` (`wordle+cl`), `wordle with critic` (`wordle+cr`), `private-shared` (`priv/sh`).

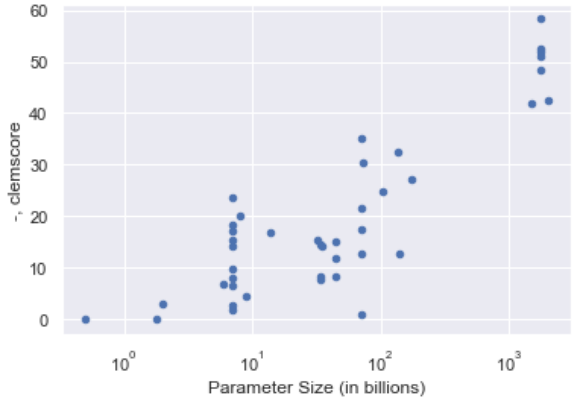

Figure 3: Plotting score vs. model size

inference engine). We also test quantised versions of models so that a full specification can, e.g., be $(\mathrm{hf}, q_{4b}(\Theta_{l3-70B-inst}))$, for access to a 4bit quantised version of llama3-70b (Touvron et al., 2023b) via Huggingface Transformers (Wolf et al., 2020). The score vector $\mathbf{s}$ consists of two scores per game (measuring, as described above, format instruction following and game play ability), separate averages of these over all games, and a combined score that is the product, or the quality weighted by the percentage of attempted game plays not aborted through formally invalid moves; Figure 2 visualises this for two models. We took `clembench` version 1.6 (Beyer et al., 2024) and ran the full benchmark on our cluster (4 x NVIDIA A100). We pair this score matrix with information about model characteristics (such as a number of parameters, size of the training corpus, and training methods), to the extent that we could reconstruct it. See Appendix for the complete list of models, quantisation, and inference methods.

4 First Observations

Let us get the obvious out of the way. Figure 3 plots (aggregated) score on `clembench` versus size of the model (in terms of number of parameters, log transformed; again with estimations where not public knowledge). The figure suggests that there might be a direct relationship, and indeed, a linear(-log) regression model finds a significant regression equation ($F(1, 18) = 23.56, p < 0.000$), with an R^2 of .57). If size of the training data is added in as a term, R^2 increases to .67 ($F(2, 17) = 17.31, p < 0.000$).

These results also indicate, however, that there is an unexplained remainder. This is brought out more clearly by Figure 5 (Appendix), which visualises the spread within various model size classes. The knowledge that many of the models within one class often are derived from the same base model gives a good starting point for further analyses (see next section).

As can be seen in Figures 4 and 6 (Appendix), while when aggregated it might look like *% played* (measuring format instruction following) and *quality* (measuring the quality of game play in successfully terminated games) develop in lockstep, when looking at individual games, one can see that this need not be the case. The ability to play, both formally and with any success, the *imagegame* (where player A instructs player B on how to draw a character-based grid-image) comes late (i.e., needs a larger model). In contrast, for *taboo* or *wordle*, the formatting instructions can be met even by smaller models, while quality increases only slowly.

Starting off of this, we look into these developments in more depth in the next section.

5 From Base Model To Chat Model

Base Model	Derived Model	Score
`mistral-7b-v0.1`	`starling-lm-7b-beta`	06.56
	`mistral-7b-instruct-v0.1`	08.01
	`openchat-3.5-0106`	17.10
	`openchat-3.5-1210`	18.22
	`openchat-3.5`	23.64
`llama-2-70b`	`llama-2-70b-chat-hf`	00.81
	`tulu-2-dpo-70b`	12.62
	`wizardlm-70b-v1.0`	17.40
	`sheep-duck-llama-2-70b-v1.1`	21.50

Table 1: Clemscore of different instruction-tuned models with the same base model

The fact that several of the models that were tested build off the same base model allows for a kind of 'natural experiment' on the influence of tuning methods. As Figure 1 shows, the scores achieved by the five derivatives from `mistral-7b-v0.1` are spread over 17 points. Where can this come from? We know what differs between these models (apart from the official `mistral-7b-instruct-v0.1` model, for which details are not available): `starling-lm-7b-beta` is trained using PPO (Ziegler et al., 2019), while the `openchat-3.5` family is fine-tuned using C-RLFT (Wang et al., 2023). Interestingly, with respect to performance on this benchmark, the `openchat-3.5` models *regressed* over time, with the newest version achieving 6.5 less than the oldest one. This might be a side-effect of the attempts by the developers to make *coding + general tasks*

capabilities and *mathematical reasoning* capabilities separately accessible through separate chat templates (Wang et al., 2023).

We can also look at derivatives of `llama-2-70b-hf`, which similarly show a wide spread of performance points. Here, we focus on the data mix. `Sheep-duck-llama-2-70b-v1.1` (Lee et al., 2023) combines Orca (Mukherjee et al., 2023) and Alpaca-inspired data,[2] `wizardlm-70b-v1.0` uses the Wizard Evol Instruct dataset (Xu et al., 2023), and `tulu-2-dpo-70b` employs the Tulu-v2 mix (Ivison et al., 2023). Also, the latter model is trained with Direct Preference Optimization (DPO) (Cui et al., 2023) using the "UltraFeedback" dataset (Rafailov et al., 2023).

These observations further highlight the importance of fine-tuning data and method (Abdin et al., 2024), showing that even small models can achieve comparatively high scores (e.g., `openchat-3.5`), provided that the right combination is found. It also highlights that generality might be harder to achieve, especially for smaller models: the `openchat-3.5` variants that perform worse here do indeed measure higher on other benchmarks (Wang et al., 2023).

6 Practicalities: How to Host an Agent

Above, we have taken care to include in a full specification of a scored model (e.g., $(\mathrm{hf}, q_{4b}(\Theta_{l3-70B-inst}))$) both details about possible weight quantisation and about the access method. One might wonder why the latter—is a model not fully defined by its weights (and the network architecture)? As Table 2 shows, perhaps surprisingly, the way of accessing a model (via API provider, locally) influences the achieved scores of around 5 clemscore points in the case of `meta-llama-3-70b-instruct`. For more discussion, see the Appendix; here, we point to one possible reason for the discrepancy: not all sampling parameters are exposed by these APIs, and hence, the actually used settings might differ.

Lastly, we found that model quantization also influences the *clemscore*, as shown in Table 3, but not dramatically so, with 8-bit quantization having only negligible impact for the examined models.[3]

Model	Backend	Score
`Meta-Llama-3-70B-Instruct`	Groq	39.34
	Together AI	35.20
	HuggingFace (local)	35.11
	Anyscale	34.26
`Meta-Llama-3-8B-Instruct`	Together AI	21.66
	HuggingFace (local)	19.99
	Anyscale	19.32
	Groq	17.79

Table 2: Clemscore using different inference methods.

Model	Quantization	Score
Meta-Llama-3-8b-instruct	None	19.99
	GGUF Q8_0	20.54
	GGUF Q4_K_M	11.75
c4ai-command-r-plus	None	24.94
	GGUF Q8_0	25.53
	GGUF Q4_K_M	19.48
Meta-Llama-3-70b-instruct	None	35.11
	GGUF Q8_0	38.88
	GGUF Q4_K_M	33.57

Table 3: Clemscores at different quantization.

Smaller models, like Meta-Llama-3-8b-Instruct, show a greater impact on clembench performance at 4-bit quantization than larger models. Overall, we consider this good news, as it seems safe to serve 8bit quantized models (at lower cost) for agentive tasks.

7 Conclusions

We use a recently introduced benchmark, `clembench`, that test situated agency capabilities through LLM self-play and relates model performance to model characteristics. We find that while model scale, both in terms of model parameter size and training data amount, accounts for a large portion of performance on the benchmark, there is a notable impact of other factors. Comparing models trained off the same base in a kind of "natural experiment", we find that the quality of the instruction-tuning data mix also appears to profoundly impact performance, as models of the same magnitude show a wide spread of performance scores on the benchmark. In particular, we find a good mix between "chat" and "reasoning" data in the fine-tuning data to be beneficial. From a more practical perspective, we also find that the inference implementation has a notable influence, while conversely milder weight quantisation has little impact; this suggests best practices in setting up models for realising situated interactions.

[2] `https://huggingface.co/Riiid/sheep-duck-llama-2-70b-v1.1`, `https://crfm.stanford.edu/2023/03/13/alpaca.html`

[3] Lower scores for the unquantized versions are likely due to the mentioned differences in sampling between HuggingFace `transformers` used for unquantized versions and `llama.cpp` used for quantized GGUF versions.

Limitations

All clembench game instances used are English only. While some of the models mentioned are able to process and generate text in other languages, our findings do not cover potential performance differences with non-English inputs.

For proprietary models accessed via remote API, information about training data and inference methods is not fully available. This limits our ability to compare them and derive conclusions from the comparison properly. See Appendix D for details.

While clembench closely approximates natural text conversations, the produced dialogues are still artificial, as are the parsing and scoring of model outputs. Thus, high clembench scores are not to be seen as indicating good end-user performance or naturalness of conversation of the examined models.

The clembench score is also not an indication of the alignment or safety of a model. Note that sometimes performance on clembench games might be at odds with safety mechanisms built into some of the models: A refusal to follow game instructions ("As an AI model, I can't...") would simply lead to a parsing failure, and a 0 'played' score for instance. However, as none of the game instructions could correctly be classified as unsafe requests, we do not consider this to be an unfair disadvantage for these models.

References

Marah I Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat S. Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Martin Cai, and et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *CoRR*, abs/2404.14219.

Anne Beyer, Kranti Chalamalasetti, Sherzod Hakimov, Brielen Madureira, Philipp Sadler, and David Schlangen. 2024. clembench$_{2024}$: A challenging, dynamic, complementary, multilingual benchmark and underlying flexible framework for llms as multi-action agents. *Preprint*, arXiv:2405.20859.

Matthew Carrigan. 2023. Chat templates.

Kranti Chalamalasetti, Jana Götze, Sherzod Hakimov, Brielen Madureira, Philipp Sadler, and David Schlangen. 2023. clembench: Using game play to evaluate chat-optimized language models as conversational agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 11174–11219, Singapore. Association for Computational Linguistics.

Tarun Kumar Chawdhury. 2024. Content moderation: An llm api with a carefully crafted system prompt is all you need.

Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Wei Zhu, Yuan Ni, Guotong Xie, Zhiyuan Liu, and Maosong Sun. 2023. Ultrafeedback: Boosting language models with high-quality feedback. *CoRR*, abs/2310.01377.

Jinhao Duan, Renming Zhang, James Diffenderfer, Bhavya Kailkhura, Lichao Sun, Elias Stengel-Eskin, Mohit Bansal, Tianlong Chen, and Kaidi Xu. 2024. Gtbench: Uncovering the strategic reasoning limitations of llms via game-theoretic evaluations. *CoRR*, abs/2402.12348.

Navanit Dubey. 2024. Update to llama-3-8b-instruct tokenizer configuration.

Samir Yitzhak Gadre, Georgios Smyrnis, Vaishaal Shankar, Suchin Gururangan, Mitchell Wortsman, Rulin Shao, Jean Mercat, Alex Fang, Jeffrey Li, Sedrick Keh, Rui Xin, Marianna Nezhurina, Igor Vasiljevic, Jenia Jitsev, Alexandros G. Dimakis, Gabriel Ilharco, Shuran Song, Thomas Kollar, Yair Carmon, Achal Dave, Reinhard Heckel, Niklas Muennighoff, and Ludwig Schmidt. 2024. Language models scale reliably with over-training and on downstream tasks. *CoRR*, abs/2403.08540.

Ran Gong, Qiuyuan Huang, Xiaojian Ma, Hoi Vo, Zane Durante, Yusuke Noda, Zilong Zheng, Song-Chun Zhu, Demetri Terzopoulos, Li Fei-Fei, and Jianfeng Gao. 2023. Mindagent: Emergent gaming interaction. *CoRR*, abs/2309.09971.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. 2022. Training compute-optimal large language models. *CoRR*, abs/2203.15556.

Hamish Ivison, Yizhong Wang, Valentina Pyatkin, Nathan Lambert, Matthew E. Peters, Pradeep Dasigi, Joel Jang, David Wadden, Noah A. Smith, Iz Beltagy, and Hannaneh Hajishirzi. 2023. Camels in a changing climate: Enhancing LM adaptation with tulu 2. *CoRR*, abs/2311.10702.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *CoRR*, abs/2001.08361.

Ariel N. Lee, Cole J. Hunter, and Nataniel Ruiz. 2023. Platypus: Quick, cheap, and powerful refinement of llms. *CoRR*, abs/2308.07317.

Jiatong Li, Rui Li, and Qi Liu. 2023. Beyond static datasets: A deep interaction approach to LLM evaluation. *CoRR*, abs/2309.04369.

Sheng Lu, Irina Bigoulaeva, Rachneet Sachdeva, Harish Tayyar Madabushi, and Iryna Gurevych. 2023. Are emergent abilities in large language models just in-context learning? *CoRR*, abs/2309.01809.

Kaifeng Lyu, Haoyu Zhao, Xinran Gu, Dingli Yu, Anirudh Goyal, and Sanjeev Arora. 2024. Keeping llms aligned after fine-tuning: The crucial role of prompt templates. *CoRR*, abs/2402.18540.

Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. 2023. Orca: Progressive learning from complex explanation traces of GPT-4. *CoRR*, abs/2306.02707.

Dan Qiao, Chenfei Wu, Yaobo Liang, Juntao Li, and Nan Duan. 2023. Gameeval: Evaluating llms on conversational games. *CoRR*, abs/2308.10032.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *CoRR*, abs/2305.18290.

Chufan Shi, Haoran Yang, Deng Cai, Zhisong Zhang, Yifan Wang, Yujiu Yang, and Wai Lam. 2024. A thorough examination of decoding methods in the era of llms. *CoRR*, abs/2402.06925.

Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R. Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, Agnieszka Kluska, Aitor Lewkowycz, Akshat Agarwal, Alethea Power, Alex Ray, Alex Warstadt, and et al. 2022. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *CoRR*, abs/2206.04615.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023a. Llama: Open and efficient foundation language models. *CoRR*, abs/2302.13971.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, and et al. 2023b. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288.

Guan Wang, Sijie Cheng, Xianyuan Zhan, Xiangang Li, Sen Song, and Yang Liu. 2023. Openchat: Advancing open-source language models with mixed-quality data. *CoRR*, abs/2309.11235.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022. Emergent abilities of large language models. *CoRR*, abs/2206.07682.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

Yue Wu, Xuan Tang, Tom M. Mitchell, and Yuanzhi Li. 2023. Smartplay : A benchmark for llms as intelligent agents. *CoRR*, abs/2310.01557.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. 2023. Wizardlm: Empowering large language models to follow complex instructions. *CoRR*, abs/2304.12244.

Xuhui Zhou, Hao Zhu, Leena Mathur, Ruohong Zhang, Haofei Yu, Zhengyang Qi, Louis-Philippe Morency, Yonatan Bisk, Daniel Fried, Graham Neubig, and Maarten Sap. 2023. SOTOPIA: interactive evaluation for social intelligence in language agents. *CoRR*, abs/2310.11667.

Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul F. Christiano, and Geoffrey Irving. 2019. Fine-tuning language models from human preferences. *CoRR*, abs/1909.08593.

A The Full Results Overview

Figure 4 shows aggregated scores and a breakdown by game for each model. Figure 5 shows the *clem-score* of each model, where the models are binned in size buckets. This illustrates the spread of achievable performance within the same size bracket.

B Dynamics

Figure 6 plots *% played* and *quality* per model, with the models sorted by size, and within the same size, by performance. That is, by going from left to right on this graph, the improvement due to size

[1]Our investigation on the observed behaviors is dependent on details marked by the model-provider/developers.

Figure 4: Profiles for all models (default inference method, not quantised), sorted by aggregated score (clemscore). For each game (and the average), %-played blue/left, quality brown/right.

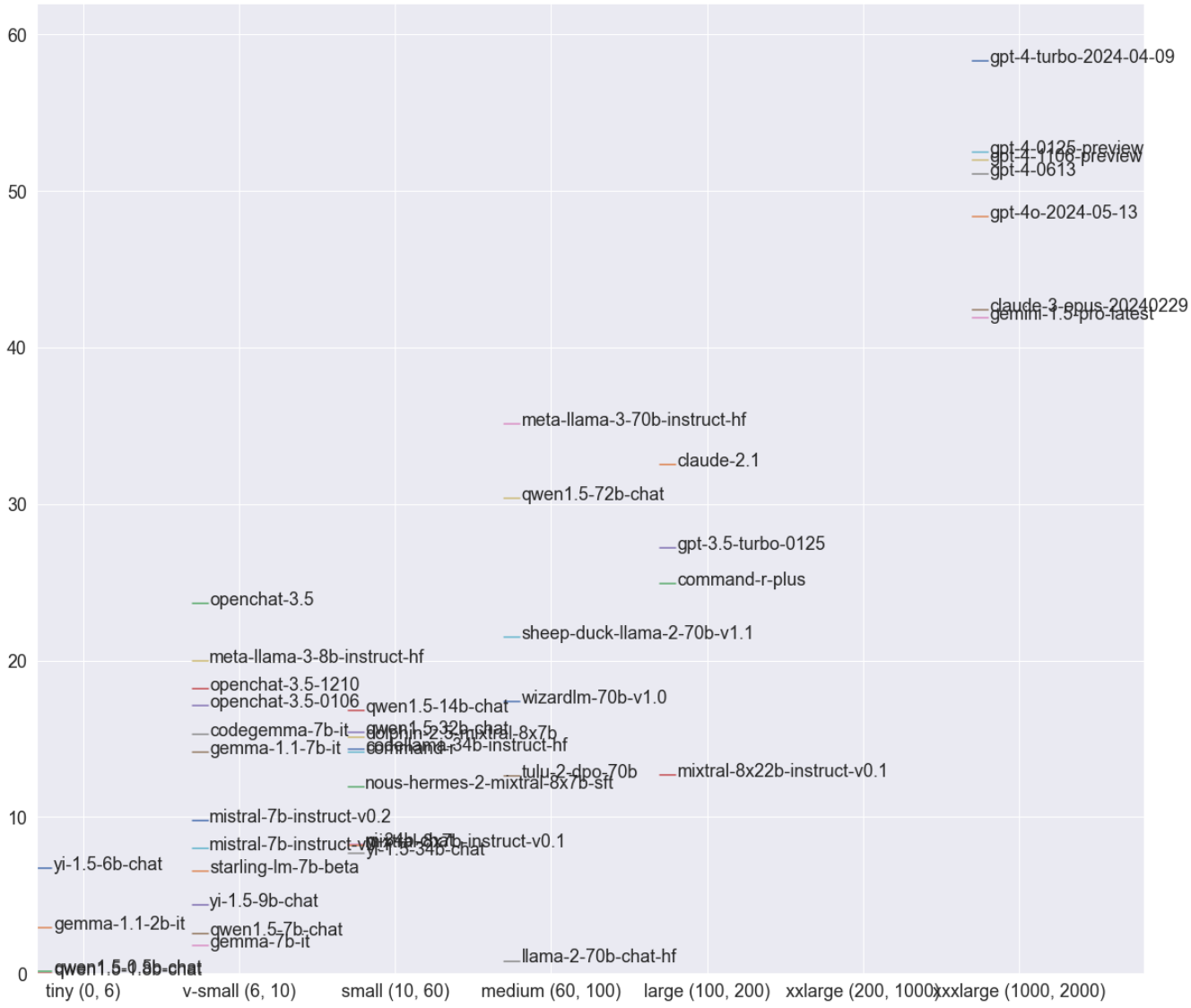

Figure 5: Performance, binned models sizes

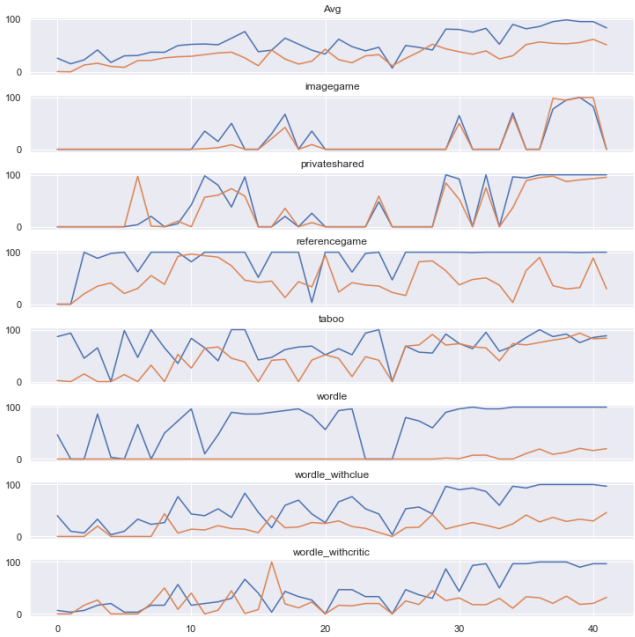

Figure 6: Dynamics of improvements. Models sorted by size (parameters) and, within same size, by aggregated score. I.e., on the left is the smallest and worst performing model, on the right the best performing of the biggest ones. Blue is % played, orange is quality score.

(or other factors) can be seen. While for the aggregated measure in the top row, both metrics show a not very steep, but steady incline, looking at the individual games reveals difference between the games. The *imagegame* sticks out as apparently requiring special capabilities (namely, being able to interpret character-sequences as "images") that are likely due to special training data and not model size. *Taboo*, on the other hand, shows a nice steady improvement. Lastly, *wordle* only begins to see modest improvements for very large models; the line suggests that here an increase of size might yield further improvements. None of these curves seem to point a sudden emergence of a capability at a certain size point.

C From Base Model to Chat Model

Generally, when a model is improvised by its developers, it is majorly aimed to allow the model to have improved capabilities with regards to overall improvements on the downstream benchmarks. However, our analysis says the opposite. As we outline this in Figure 3, `openchat-3.5` (Jun 2023) has a better clemscore compared to its successor counterparts i.e. `openchat-3.5-1210` (Dec 2023), and `openchat-3.5-0106` (Jan 2024). All three models are fine-tunes of the same base model, i.e. `mistral-7b-v0.1`.

Is model improvement across benchmarks proportional to improvement on agentive tasks? If so, are there additional factors that can alter this relationship?[1]

Below, we analyze potential factors responsible for the observed deviations:

First, looking into the openchat series of models, they are instruction tuned using the C(Conditioned)-RLFT technique (Wang et al., 2023), which eases the need for high-quality preference optimization datasets—and uses SFT training data, consisting of a small amount of expert data without any preference labels. So, to put it aside, all of these models utilize a similar tuning technique, thus it should be fine to disregard the factor of tuning method from the discussion.

Next, selection of training data, might also have an effect on the models' performance. Before that, we would like to emphasize on the formatting of input data for either of these models. Unlike `openchat-3.5`, which uses a single prompt format for all training inputs, later versions are trained to handle two different formats or "modes": one for general reasoning tasks and another for mathematical reasoning. All three models are mostly trained on similar data, which is mainly synthetic multi-turn conversations (see table 5). Apparently, the key difference is that the later models are also trained to evaluate and give feedback on responses.

Conclusively, even though this can be regarded as speculation, based on the observations indicated through the above discussion, it would be ideal to assert that an increase in model competencies inversely affects precision on tasks at hand (i.e., individual dialogue games). Smaller models can compress information to a certain extent, and overloading them with more capabilities may reduce proficiency.

More importantly, in clembench, for individual game instances, which seek an ensemble of abilities from the testing agent, it is inefficient to identify them explicitly beforehand. This comes as a drawback, since models like `openchat-3.5-1210`, and `openchat-3.5-0106` (available under the category of `v-small` models) are trained to process abilities like general reasoning and mathematical reasoning separately.

Base Model	Model Name	Model Provider	Release Date	cs	% pl	q sc	Instruction Tuning Data
gemma-7b	gemma-7b-it	Google	2024-02-01	1.82	17.78	10.23	NaN
gemma-7b	gemma-1.1-7b-it	Google	2024-04-01	14.14	49.67	28.46	NaN
gemma-7b	codegemma-7b-it	Google	2024-04-01	15.30	51.95	29.45	NaN
llama-2-70b-hf	llama-2-70b-chat-hf	Meta	2023-07-01	0.81	7.14	11.31	NaN
llama-2-70b-hf	tulu-2-dpo-70b	Allenai	2024-11-01	12.62	49.76	25.37	ultrafeedback-binarized, tulu-v2-sft-mixture
llama-2-70b-hf	wizardlm-70b-v1.0	Wizardlmteam	2023-08-01	17.40	46.19	37.66	wizardlm-evol-instruct-v2-196k
llama-2-70b-hf	sheep-duck-llama-2-70b-v1.1	Riid	2023-09-01	21.50	41.19	52.20	orca and alpaca inspired data
mistral-7b-v0.1	starling-lm-7b-beta	Nexusflow	2024-03-01	6.56	30.89	21.25	nectar
mistral-7b-v0.1	mistral-7b-instruct-v0.1	Mistralai	2023-09-01	8.01	37.14	21.58	NaN
mistral-7b-v0.1	openchat-3.5-0106	Openchat	2024-01-01	17.10	52.57	32.52	sharegpt, openorca, capybara, goat, glaive, metamathqa, mathinstruct, oasst1, feedback-collection
mistral-7b-v0.1	openchat-3.5-1210	Openchat	2023-12-01	18.22	51.19	35.60	sharegpt, openorca, capybara, goat, glaive, metamathqa, mathinstruct, oasst1, feedback-collection
mistral-7b-v0.1	openchat-3.5	Openchat	2023-10-01	23.64	63.52	37.22	sharegpt, openorca, capybara, goat, glaive, metamathqa, mathinstruct, oasst1
mixtral-8x7b-v0.1	mixtral-8x7b-instruct-v0.1	Mistralai	2023-12-01	8.17	47.62	17.15	NaN
mixtral-8x7b-v0.1	nous-hermes-2-mixtral-8x7b-sft	NousResearch	2024-01-01	11.95	39.68	30.12	openhermes-2.5
mixtral-8x7b-v0.1	dolphin-2.5-mixtral-8x7b	cognitivecomputations	2023-12-01	15.10	46.38	32.55	magicoder-oss-instruct-75k, magicoder-evol-instruct-110k, openhermes

Table 4: Comparing models trained off the same base model

Dataset Name	Links
capybara	https://huggingface.co/datasets/LDJnr/Capybara
feedback-collection	https://huggingface.co/datasets/prometheus-eval/Feedback-Collection
glaive	https://huggingface.co/datasets/glaiveai/glaive-code-assistant
goat	https://huggingface.co/datasets/tiedong/goat
magicoder-evol-instruct-110k	https://huggingface.co/datasets/ise-uiuc/Magicoder-Evol-Instruct-110K
magicoder-oss-instruct-75k	https://huggingface.co/datasets/ise-uiuc/Magicoder-OSS-Instruct-75K
mathinstruct	https://huggingface.co/datasets/TIGER-Lab/MathInstruct
metamathqa	https://huggingface.co/datasets/meta-math/MetaMathQA
nectar	https://huggingface.co/datasets/berkeley-nest/Nectar
oasst1	https://huggingface.co/datasets/OpenAssistant/oasst_top1_2023-08-25
openhermes	https://huggingface.co/datasets/teknium/openhermes
openhermes-2.5	https://huggingface.co/datasets/teknium/OpenHermes-2.5
openorca	https://huggingface.co/datasets/Open-Orca/OpenOrca
sharegpt	https://huggingface.co/datasets/openchat/openchat_sharegpt4_dataset
tulu-v2-sft-mixture	https://huggingface.co/datasets/allenai/tulu-v2-sft-mixture
ultrafeedback-binarized	https://huggingface.co/datasets/HuggingFaceH4/ultrafeedback_binarized
wizardlm-evol-instruct-v2-196k	https://huggingface.co/datasets/WizardLMTeam/WizardLM_evol_instruct_V2_196k

Table 5: List of instruction tuning datasets

Model	API Provider / Backend	Clemscore	Avg. % Played	Avg. % Quality Score	Link
Meta-Llama-3-70B-Instruct	HuggingFace (local)	35.11	80.72	43.5	https://huggingface.co/meta-llama/Meta-Llama-3-70B-Instruct
	Groq	39.34	82.35	47.77	https://groq.com/
	Anyscale	34.26	80.00	42.82	http://anyscale.com/
	Together AI	35.20	79.52	44.26	https://www.together.ai/
Meta-Llama-3-8B-Instruct	HuggingFace (local)	19.99	76.1	26.27	https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct
	Groq	17.79	77.43	22.98	https://groq.com/
	Anyscale	19.32	75.81	25.48	http://anyscale.com/
	Together AI	21.66	74.67	29.01	https://www.together.ai/

Table 6: Inference results of the used models (specific to API Provider/Backend) on the clembench. Reports for each model the clemscore, with average % played, and average % quality score. For models acquired through HuggingFace, the corresponding repository is linked, for those accessed via a (remote) inference interface, the API provider's website.

Model Name	File Format	Clemscore	Avg. % Played	Avg. % Quality Score	Link
meta-llama-3-8b-instruct-gguf-q4	GGUF Q4_K_M	11.75	54.84	21.43	https://huggingface.co/MaziyarPanahi/Meta-Llama-3-8B-Instruct-GGUF
meta-llama-3-8b-instruct-gguf-q8	GGUF Q8_0	20.54	69.05	29.74	https://huggingface.co/MaziyarPanahi/Meta-Llama-3-8B-Instruct-GGUF
meta-llama-3-8b-instruct-hf	safetensors	19.99	76.1	26.27	https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct
c4ai-command-r-plus-gguf-q4	GGUF Q4_K_M	19.48	66.43	29.33	https://huggingface.co/pmysl/c4ai-command-r-plus-GGUF
c4ai-command-r-plus-gguf-q8	GGUF Q8_0	25.53	74.24	34.39	https://huggingface.co/pmysl/c4ai-command-r-plus-GGUF
command-r-plus	-	24.94	74.9	33.3	https://cohere.com/
meta-llama-3-70b-instruct-gguf-q4	GGUF Q4_K_M	33.57	78.33	42.86	https://huggingface.co/MaziyarPanahi/Meta-Llama-3-70B-Instruct-GGUF
meta-llama-3-70b-instruct-gguf-q8	GGUF Q8_0	38.88	74.4	52.26	https://huggingface.co/MaziyarPanahi/Meta-Llama-3-70B-Instruct-GGUF
meta-llama-3-70b-instruct-hf	safetensors	35.11	80.72	43.5	https://huggingface.co/meta-llama/Meta-Llama-3-70B-Instruct

Table 7: Inference results of the used models (specifically quantized or with normal floating precision) on the clembench. Reports for each model clemscore, with average % played, and average % quality score. For models acquired through HuggingFace, the corresponding repository is linked, for those accessed via a (remote) inference interface, the API provider's website.

D Inference

Table 6 shows the surprising spread of scores achieved by what should be the same model, when accessed via different providers.

Three main factors might lead to these differences:

Input formatting, as in the applied chat template (Carrigan, 2023) or additions to the initial user or system message (Chawdhury, 2024; Lyu et al., 2024), can strongly impact the produced replies. As 'OpenAI-compatible' remote APIs only receive an array of message objects, we could not control for the actual input contexts. For our local inference, the method of creating input contexts was kept constant.

Sampling parameters, with only a subset commonly exposed via remote API, can also influence replies (Shi et al., 2024) and are commonly tweaked to align better with end-user expectations. While we controlled for temperature, setting it to zero for local inference and remote APIs, other common sampling parameters like repetition penalties were left to default values supplied by model creators or API providers.

Lastly, inference features pertaining to **turn-taking** can have great influence on clembench 'played' scores, such as the proper production and processing of end-of-turn tokens to stop further text generation[4]. A prominent example of this are the recent Meta-Llama-3-Instruct models, the highly popular HuggingFace implementation of which had the end-of-text token, used mainly for non-instruct uses and training, set as the stop token instead of the end-of-turn token, leading to overly long replies and quick deterioration of the generated text (Dubey, 2024).

The inaccessibility of many of these inference parameters on commercial remote APIs, along with proprietary secrecy about unexposed parameters and training, make it hard to argue about their performance effects beyond model scale as mentioned in Section 4. There is also a noticeable sparsity of literature examining the effects of these factors on benchmark scores, with API parameter inaccessibility likely exacerbated by prohibitively high local computational demands posed by the stochastic nature of sampling-based generation.

Table 7 shows results at different quantisation strengths.

[4]Clembench uses tokens predefined by model developers to stop text generation, and while these are metadata, we consider them to be part of model architecture and inference implementation. Generation stopping is important for user experience and saves resources, thus this sensitivity is desirable over the use if custom stopping measures by a benchmark.